# תרגיל מסכם – לינקינג פארק

## ברק גונן

ברוכים הבאים לפארק הלינקינג, התרגיל המסכם בסביבת Windows. בפארק הזה, המשחקים הם שיטות שונות לביצוע code linking. נעסוק ב-Static Linking, ב-Dynamic Linking, נלמד Run Time Linking ו-Load Time Linking. הרבה לינקינג יהיה בפארק שלנו! מה יהיה בסוף? נאפשר לתוכנה לבצע דברים שהמתכנתים שלה לא תיכננו שהיא תבצע. לדוגמה, נגרום ל-excel לפתוח חלון חדש בו יודפס השם שלנו. השיטה שנשתמש בה נקראת DLL Injection, או "הזרקת DLL".

לפני שניגש לעבוד, כמה מילים על ביצוע הזרקת DLL והאם זה בכלל חוקי. להזרקת DLL יש שימושים טובים ושימושים רעים, הכל תלוי במה בדיוק אנחנו מבצעים איתה. אם אנחנו יכולים לגרום לתוכנה לבצע דברים שהמתכנתים שלה לא תיכננו, הדבר הראשון שעולה במחשבה הוא שימוש זדוני: ניקח תוכנה תמימה ונהפוך אותה לנוזקה. כיוון שאף אחד לא יחשוד בתוכנה תמימה ומוכרת, זו טכניקה מקובלת על כותבי נוזקות. לעומת זאת, אנחנו יכולים לגרום לתוכנה להוציא פלט של כל הקבצים שהיא פתחה, המקומות ברגיסטרי שניגשה אליהם, ה-Mutexים שמשתמשת בהם. דבר זה יכול לאפשר לנו לדבג את התוכנה ולוודא שהיא עובדת כשורה, לכן זו גם טכניקה מקובלת אצל כותבי תוכנות אנטיוירוס. בשורה התחתונה, הזרקת DLL אינה טובה או רעה, השאלה היא מה מזריקים. מה שבטוח- זו משימה מעניינת ומאתגרת!

כיוון שהמשימה היא די מורכבת, נחלק אותה לכמה חלקים:

1. נלמד מהי library ומהו Static Linking?
2. נלמד מהו DLL
3. נלמד כיצד לייבא DLL ב-Load Time
4. נלמד כיצד לייבא DLL ב-Run Time
5. נלמד כיצד לבצע DLL Injection באמצעות הרגיסטרי
6. בחלק המתקדם ביותר, נסקור שיטה שמבצעת DLL Injection באמצעות פתיחת thread בתוך process אחר, ובאופן מכוון לא נסביר את כל הפרטים אלא נותיר חלקים למחקר עצמי

מהי library ומהו Static Linking?

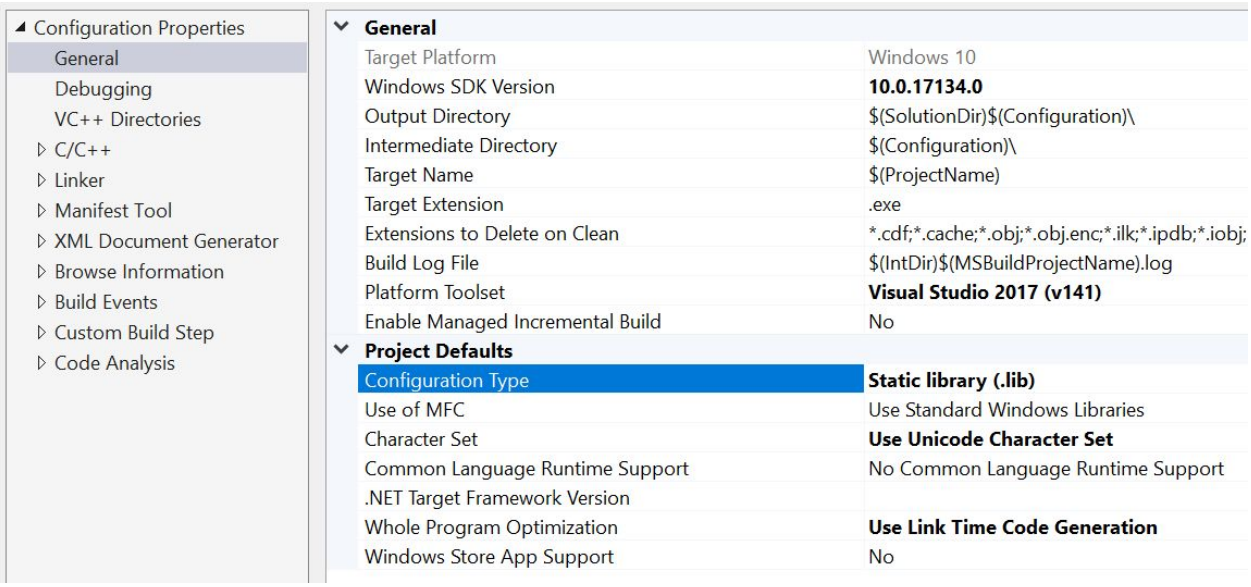
המונח library מציין קוד מקומפל, שעדיין לא עבר את השלב האחרון הנדרש להפיכתו לקוד הרצה exe. הקוד של ה-library הוא בשפת מכונה, אך המעבד לא יכול להריץ אותו כיוון שלא בוצע לו linking- הכתובות שרשומות בקוד

(פונקציות, משתנים) אינן נכונות. אם יש ברשותנו library כלשהי, אנחנו יכולים להפוך אותה לקוד הרצה באמצעות פעולת linking עם קוד אחר שיש לנו. כך יהפוך ה-library לחלק מקוד התוכנית שלנו, ממש כאילו כתבנו אותו בעצמנו.

הצורך ב-library נובע מכך שיש קטעי קוד שהם שימושיים למספר רב של תוכנות. לדוגמה, קוד שיודע לקלוט היכן המשתמש הקליק עם העכבר על המסך, יהיה שימושי לכל תוכנה שיש לה חלון משתמש. לכן כל מי שמתכנת תוכנה שיש לה חלון משתמש ירצה לשלב את הקוד הזה בתוכנה שלו. אפשרות אחת היא לקחת את הקוד שמטפל בעכבר ולקמפל אותו יחד עם קוד התוכנה. כך ניצור קובץ הרצה EXE שיש בו כבר את כל הפונקציות שמטפלות בעכבר.

אולם בגישה זו יש בעיה: אנחנו צריכים את קוד המקור. ייתכן שחברת התוכנה שפיתחה את הקוד הזה לא תרצה לספק את קוד המקור. אחרי הכל, מה מונע מאיתנו לעשות עם הקוד מה שנרצה לאחר שהשגנו אותו?

על הבעיה הזו עונה שימוש בקבצי lib. היכנסו אל visual studio ופיתחו פרוייקט כלשהו. שנו את הגדרות הפרוייקט כך שהתוצר שלו יהיה קובץ lib. כאשר תבצעו build, תיצרו קובץ lib, שכולל את שפת המכונה של הפקודות שתכנתתם אך עדיין לא מקושר (linked) אל קבצים אחרים, כלומר הכתובות בזיכרון שנמצאות בקובץ ה-lib עדיין אינן סופיות אלא צפויות להשתנות בתהליך ה-linking. את קובץ ה-lib שיצרתם אתם יכולים למסור לכל מי שתרצו שיעשה בו שימוש, והוא יבצע linking אל הקוד שלו. צורה זו של linking נקראת **static linking**. היתרון שלה הוא שהוא מאפשר לנו לחלוק חבילות קוד, בלי לחלק את קוד המקור שעמלנו רבות על פיתוחו. בעבר, זו היתה הצורה היחידה של שיתוף קוד. אולם static linking עדיין אינו מושלם, כפי שמיד נראה.




*שינוי הגדרות הפרוייקט כך שיווצר קובץ lib*

## מהו DLL?

בחלק זה נענה על השאלה מהו DLL ומדוע יש בו צורך?

השם DLL הוא ראשי תיבות של Dynamic Link Library. הפורמט של קבצי DLL הומצא על ידי מייקרוסופט, אם כי הרעיון הכללי נמצא גם במערכות הפעלה אחרות. בלינוקס יש קבצים שמשמשים לתכלית דומה, אך הם נקראים קבצי SO, קיצור של shared object. הצורך ב-Dynamic Linking נובע מכך שישנן בעיות ש-Static Linking אינו פותר.

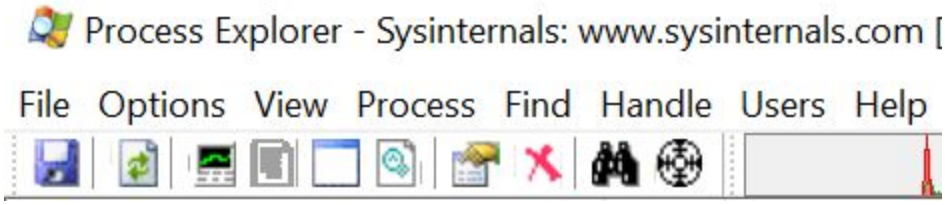
הבעיה הראשונה ב-static linking היא קושי בתחזוקה של קוד. נניח שחברת התוכנה שכותבת קוד שמטפל בעכבר שיפרה את הקוד. כעת כל מי שרוצה ליהנות מהשיפור צריך לבצע Linking מחדש של התוכנה שלו עם ה-library המשודרגת ולהפיץ לכל המשתמשים שלו את העדכון. מסובך! כמה טוב היה אם היה מספיק רק לעדכן קובץ, שכולל את הקוד המשודרג של העכבר, וכל התוכנות במחשב שמשתמשות בקובץ הזה יקבלו מיידית את היכולות המשודרגות.

הבעיה השניה היא בזבוז של זיכרון. אם עשר תוכנות נמצאות בזיכרון שלנו, וכולן עושות שימוש בעכבר, אז הפונקציות של העכבר משוכפלות עשר פעמים בזיכרון. מדוע שהקוד שמטפל בעכבר לא ייטען פעם אחת לזיכרון, וכל מי שזקוק לו יקרא אותו משם?

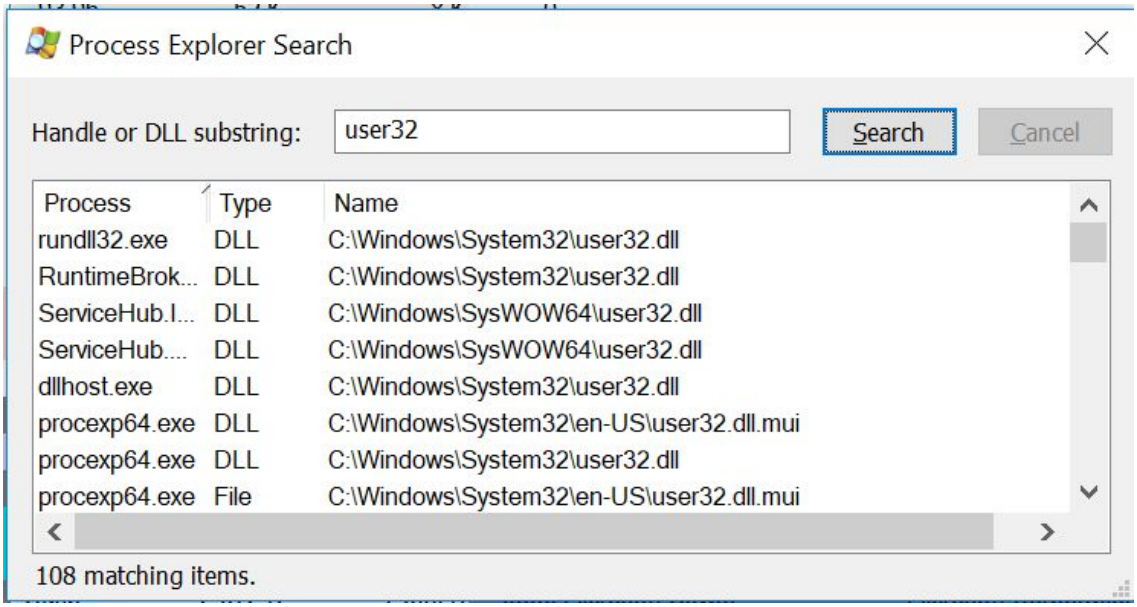
זהו היתרון של DLL. זהו קוד, שמכיל פונקציות שונות. הפונקציות הללו נטענות לזיכרון פעם אחת, וכל התוכנות שרצות במקביל משתמשות באותו עותק שנמצא בזיכרון. לכן, מספיק שיש לנו עותק אחד של ה-DLL על המחשב, שנטען לזיכרון פעם אחת. אם יש לנו עדכונים לתוכנה, נפיץ DLL חדש, כל מי שמשתמש בתוכנה שלנו יחליף את ה-DLL הישן, וכל התוכנות שעושות בו שימוש יקבלו מיד את היכולות החדשות.

הדוגמה שניתנה לפני כן, של קוד שמפעיל את העכבר, לא היתה מנותקת ממה שמתרחש באמת. פונקציות שמנהלות את העכבר, כמו פונקציות נוספות שעובדות מול חלונות פתוחים, נמצאות בקובץ user32.DLL. בואו נפתח process explorer ונבדוק מי משתמש בקובץ זה.

היכנסו ל-process explorer והקליקו על find




כעת הזינו לשורת החיפוש user32




כפי שאתם רואים, בזמן כתיבת מסמך זה, לא פחות מ-108 processים השתמשו בו זמנית ב-user32.DLL! כעת יותר ברורה התועלת בשימוש ב-DLL שנטען פעם אחת לזיכרון.

קיימות שתי דרכים לטעון DLL לתוך תוכנה- load time dynamic linking ו-run time dynamic linking.

ב- load time dynamic linking ה-DLL המבוקש נטען עם טעינת התוכנה. למה הכוונה? בניגוד ל-static linking, הקוד של ה-exe שנוצר **אינו** כולל את קוד ה-library. קוד ה-library נשאר בקובץ נפרד, מסוג DLL. יכול להיות exe שמשתמש ב-DLL שגודלו 1MB, ועדיין גודלו של ה-exe יהיה מספר KB בודדים.

איך מתבצע ה-linking בין ה-exe וה-DLL, שהינם שני קבצים נפרדים? לכל קובץ הרצה exe יש חלק, שבו כתובים כל ה-DLLים שצריך לטעון לזיכרון לפני שהמעבד מתחיל להריץ את התוכנה עצמה. לכן השיטה בה הטעינה של ה-DLLים שרשומים בקובץ ה-exe נטענים בזמן טעינת ה-exe, נקראת **load** time dynamic linking. בזמן הטעינה, ה-DLL ממופה לתוך הזיכרון הוירטואלי של ה-process שנוצר, כך שניתן יהיה לקרוא לפונקציות שלו. שימו לב שהקוד ייטען אל ה-RAM רק כאשר ה-process ישתמש בו בפועל. ובכל מקרה ה-DLL ייטען אל ה-RAM רק פעם אחת, וכל מי שמשתמש בו יעשה שימוש בעותק היחיד הטעון ל-RAM.

ב-run time dynamic linking הרעיון הוא מאד דומה, אך בהבדל אחד מרכזי: אמרנו שקובץ ה-exe כולל את רשימת ה-DLLים שיש למפות לזיכרון בזמן הטעינה. כצפוי, DLL שאינו נמצא ברשימה זו לא ייטען. אבל אפשר שבזמן הריצה ייקראו פונקציות שיטענו בעצמן את ה-DLL הנדרש. השיטה שבה ה-DLL נטען בזמן ריצה נקראת **run** time dynamic linking.

בסעיפים הבאים נלמד כיצד יוצרים DLL וכיצד טוענים אותו, בשתי השיטות.

<u>יצירת DLL</u>

נפתח פרוייקט חדש, בדוגמה בחרנו לקרוא לו mydll.

בתור התחלה, נוסיף לפרוייקט שלנו קובץ h חדש (בתוך solution explorer קליק ימני על header files ואז add, בחרו new item ולאחר מכן header file). נקרא לקובץ mydll.h. נכתוב לתוכו את הקוד הבא:

```
#pragma once

#ifdef DLL_EXPORT
#define DECLDIR __declspec(dllexport)
#else
#define DECLDIR __declspec(dllimport)
#endif

extern "C"
{
    DECLDIR void Share();
    void Keep();
}
```

מומלץ לקרוא על ההנחיות לקומפיילר pragma once, ifdef ו-"extern "C. זהו ידע חשוב אך הוא מחוץ למוקד של מסמך זה ולכן הוא לקריאה עצמית. נתמקד בשתי הפונקציות שמוגדרות- Share ו-Keep.

לפני הכל, חשוב להבין שכאשר אנחנו מפיצים DLL הכולל פונקציות שונות, ישנן פונקציות שנרצה שלא ייעשה בהן שימוש על ידי מי שאנחנו מוסרים לו את ה-DLL. אם הקוד שלנו כולל פונקציות בעלות יכולות מיוחדות, שאנחנו רוצים שיישארו רק ברשותנו, נצטרך דרך לסמן אילו פונקציות פתוחות לשימוש של כולן ואילו לא.

לפונקציה Share קודמת ההגדרה DECLDIR, שהפרוש שלה משתנה אם מוגדר DLL_EXPORT או לא. בהנחה שכן, אז הקידומת תפורש כ- __declspec(dllexport), כלומר הפונקציה **תיוצא** על ידי ה-DLL. המשמעות של זה היא שה-DLL יחשוף את הפונקציה הזו לשימוש של כל מי שמבצע linking ל-DLL. לעומתה, הפונקציה Keep היא בשום מקרה לא מיוצאת, ולכן הקוד היחיד שיכול להשתמש בה הוא קוד של ה-DLL עצמו.

לאחר שהגדרנו את קובץ ה-header, נעבור לקובץ הראשי- mydll.cpp.

```
#include "pch.h"
#define DLL_EXPORT
#include "mydll.h"

extern "C"
{
    DECLDIR void Share()
    {
        printf("I am an exported function, can be called outside the DLL\n");
    }

    void Keep()
    {
        printf("I am not exported, can be called only within the DLL\n");
    }
}
```

```
BOOL APIENTRY DllMain(HANDLE hModule,        // Handle to DLL module
                      DWORD ul_reason_for_call,
                      LPVOID lpReserved)     // Reserved
{
    switch (ul_reason_for_call)
    {
    case DLL_PROCESS_ATTACH:
        // A process is loading the DLL.
        Share();
        Keep();
        break;

    case DLL_THREAD_ATTACH:
        // A process is creating a new thread.
        break;

    case DLL_THREAD_DETACH:
        // A thread exits normally.
        break;

    case DLL_PROCESS_DETACH:
        // A process unloads the DLL.
        break;
    }
    return TRUE;
}
```

בתחילת הקובץ ניתנות מספר הנחיות לקומפיילר:

1. Include לקובץ pch.h- אותו כבר הכרנו (משמש ל-include לקבצים שאינם משתנים וכך חוסך זמן קומפילציה)
2. הגדרת DLL_EXPORT, שהיא משמעותית לטובת ההגדרות בקובץ mydll.h ולכן הכרחי שתופיע לפניו. שימו לב לא לשים את ההגדרה לפני ה-include ל-pch.h- איננו יודעים באיזה מקום עלול להיות תנאי שקשור להנחיה זו!
3. Include לקובץ mydll.h

לאחר מכן מוגדרות הפונקציות Share ו-Keep, שמה שהן מבצעות מובן מאליו.

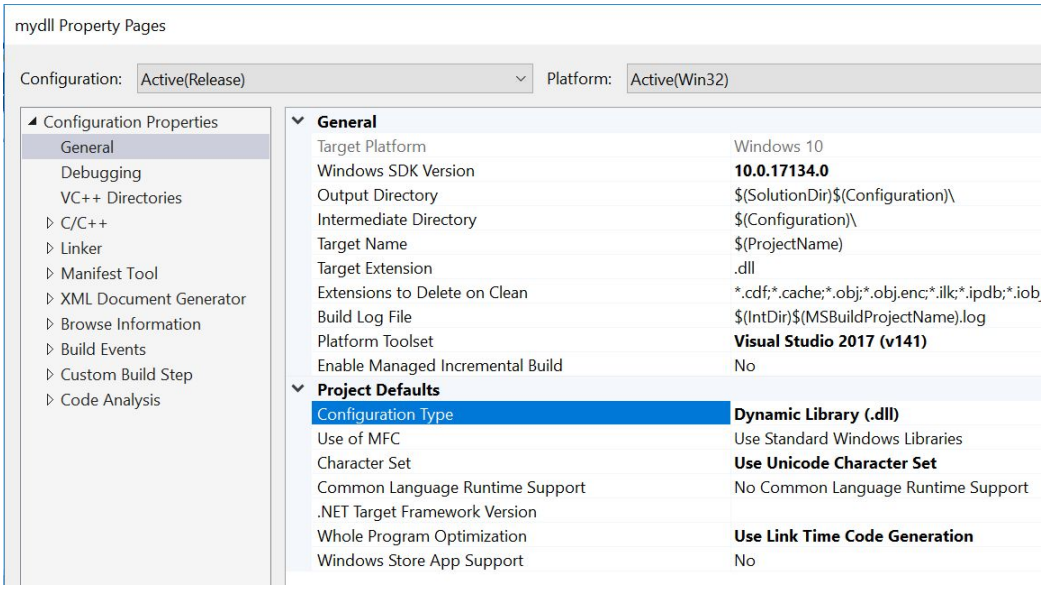
לאחר מכן מגיע החלק המעניין- dllmain. זוהי נקודת הכניסה לתוך ה-DLL שלנו. אם יש קוד שאנחנו רוצים שירוץ אוטומטית, זה המקום להכניס אותו. חשוב להבין: קוד שנמצא ב-dllmain ירוץ גם בלי שהתוכנה הראשית, שטוענת את ה-DLL, תקרא לו. מערכת ההפעלה קוראת ל-dllmain והיא עושה זאת במספר מצבים:

1. Process טען את ה-DLL שלנו
2. ה-Process שטען את ה-DLL שלנו יוצר thread חדש
3. Thread שה-DLL שלנו היה טעון בו, מסיים ריצה
4. Process שטען את ה-DLL שלנו מבצע unload, הסרת ה-DLL שלנו

כל אחד מהמקרים האלה מעביר את השליטה ל-dllmain. ההגיון בכך, הוא שאם תוכנה מסויימת טוענת את ה-DLL שלנו, ייתכן ונרצה להריץ פעולות מקדימות כלשהן, שיאפשרו לפונקציות שלנו לרוץ היטב. לדוגמה, בדיקה מהי החומרה שאנחנו רצים עליה, הקצאת זיכרון וכו'. גם בסיום ריצה או הסרת ה-DLL שלנו ייתכן שיש פעולות שאנחנו צריכים לבצע, לדוגמה שחרור זיכרון, שמירת מידע לקובץ ועוד. חשוב מאד לשים לב שמייקרוסופט מגבילים מאד את הקוד שניתן להריץ אוטומטית מ-dllmain, דברים מסויימים עלולים לגרום לקריסת ה-process שטען את ה-DLL.
מידע נוסף על מה אפשר ואי אפשר לבצע, נמצא בלינק
https://docs.microsoft.com/en-us/windows/desktop/dlls/dynamic-link-library-best-practices

בקוד הדוגמה שלנו, אנחנו משתמשים רק באפשרות להריץ קוד עם טעינת ה-DLL. מתבצעת קריאה לפונקציה Share ולאחר מכן קריאה לפונקציה Keep, שנועדה להמחיש שהקוד של ה-DLL יכול לקרוא ל-Keep ללא קושי. הפונקציה Keep חסומה רק לקריאות מקוד חיצוני.

נותר לנו רק לייצר את ה-DLL. כדי לעשות זאת, נשנה את הגדרות הקונפיגורציה של הפרוייקט שלנו, כך שבמקום EXE יווצר DLL:




*שינוי הגדרות הפרוייקט כך שיווצר DLL*

הידד! יש לנו DLL. אם נפתח את תיקיית ה-release של הפרוייקט mydll, נוכל למצוא שם את הקובץ mydll.dll וכן את הקובץ mydll.lib. מדוע נוצר לנו קובץ lib? הרי אנחנו מבצעים Dynamic Linking, ואמרנו שקבצי lib משמשים עבור Static Linking? התשובה היא, שבניגוד ל-Static Linking, קובץ ה-lib שנוצר לנו אינו כולל קוד, אלא רק אזכור של הפונקציות שה-DLL מייצא. כאשר אנחנו יוצרים קובץ EXE שמשולב בו DLL, ה-linker משתמש בקובץ ה-lib כדי לשתול בקובץ ההרצה הפניות מתאימות אל ה-DLL, כך שכל פעם שנקרא לפונקציה מתוך ה-DLL תבוצע קפיצה אל הקוד שב-DLL. תהליך זה מאפשר לנו לקרוא לפונקציות שב-DLL ממש כאילו הן פונקציות של הקוד שלנו. מיד נראה כיצד משתמשים בקובץ ה-lib.

<u>יבוא DLL ב-Load Time</u>

ניצור פרוייקט חדש, בדוגמה שלנו usemydll.

נערוך את הקובץ usemydll.cpp באופן הבא:

```
#include "pch.h"

extern "C" __declspec(dllimport) void Share();
// Other option would be-
// #include "mydll.h"

int main()
{
    Share();
    return 0;
}
```

מבין שתי הפונקציות המוגדרות ב-DLL, אפשר לקרוא מהקוד שלנו רק לפונקציה Share. כזכור, הפונקציה Keep אינה מיוצאת. כדי לשלב את הפונקציה Share אנחנו צריכים למסור לקומפיילר מידע על המבנה שלה. את המידע הזה אפשר למסור בשתי דרכים. דרך אחת היא לבצע include לקובץ mydll.h. זוהי הדרך הקלה, אך היא דורשת שהקובץ mydll.h יהיה ברשותנו. לא בטוח שזה יהיה המצב. הדרך השניה היא, כפי שנכתב בקוד שלנו, לתת לקומפיילר את המבנה של Share ולומר לו שזוהי פונקציה מיובאת.

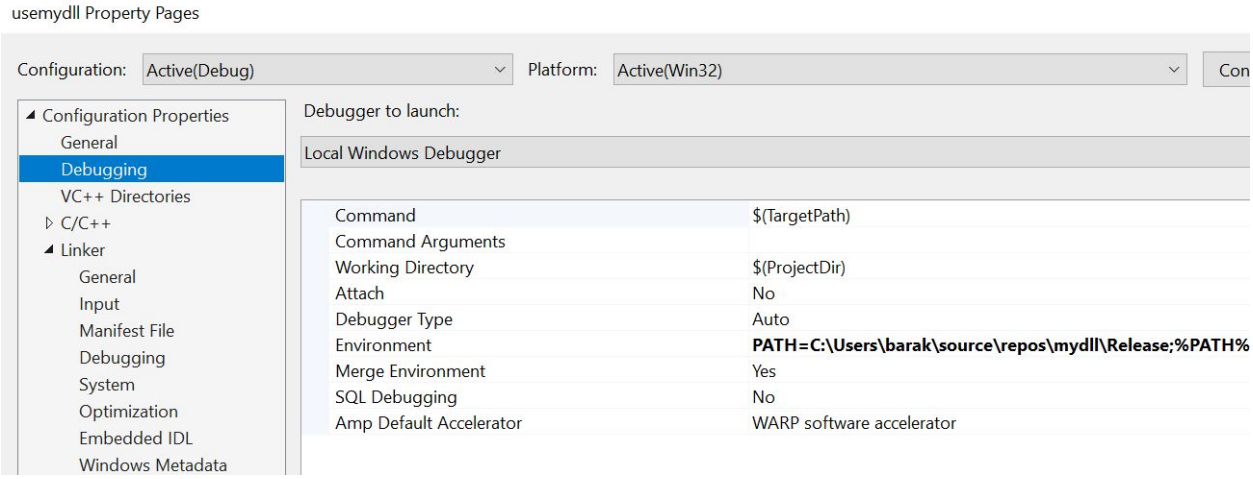
נותר לנו רק לשנות את הגדרות הפרוייקט.

כדי להודיע ללינקר היכן הוא יכול למצוא את mydll.dll, נכניס את התיקיה של mydll.dll לתוך משתנה הסביבה PATH. בתוך Environment נכתוב:

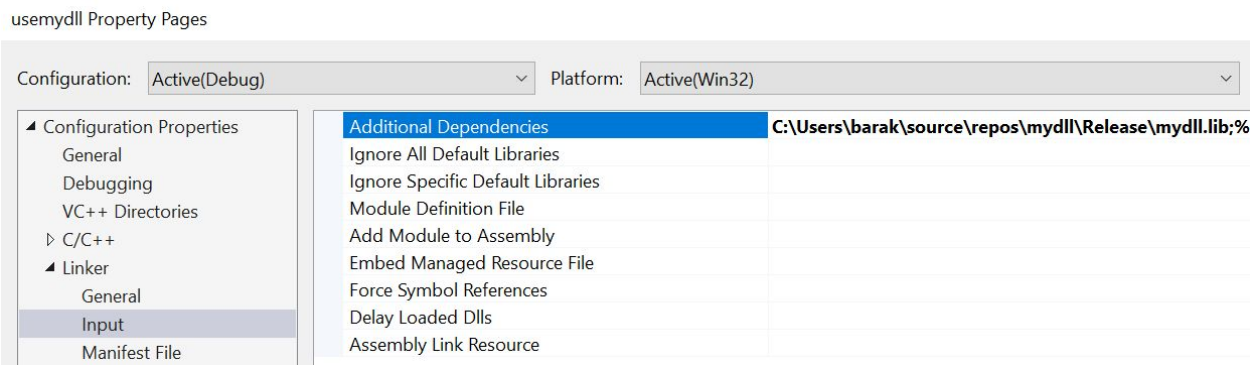
PATH=mydll_path;%PATH%

כאשר mydll_path האו הנתיב המלא אל ה-DLL שלנו. לדוגמה

PATH=c:\software\dlls;%PATH%



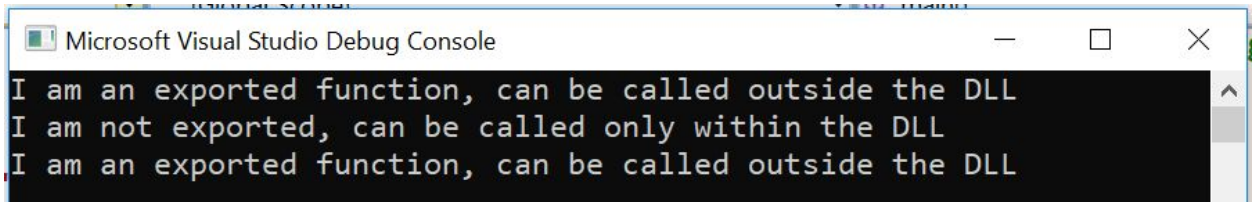
בשלב האחרון אנחנו צריכים לסייע ללינקר למצוא את קובץ ה-mydll.lib lib. נערוך את השדה Additional Dependencies כך שיכלול את שם ה-lib ואת הנתיב המלא אליו:




זהו!

כעת נריץ את התוכנית usemydll. חישבו, מה יודפס למסך? התשובה בעמוד הבא.

בעקבות הרצת usemydll תופיע ההדפסה הבאה:




מה גרם להדפסת הפלט הזה?

שתי השורות הראשונות הן תוצאה של טעינת ה-DLL. כזכור, בתוך dllmain יש קריאה ל-Share ול-Keep. השורה השלישית היא תוצאה של הקריאה ל-Share מתוך הקוד של usemydll.

סיימנו ללמוד על Load Time Dynamic Linking. כעת נוכל ללמוד על ביצוע Run Time Linking.

## יבוא DLL ב- Run Time

בניגוד ל-Load Time Dynamic Linking, כעת אנחנו לא מוסרים ללינקר שלנו פרטים על ה-DLL. אילו פרטים העברנו ללינקר ב-Load Time וכעת חסרים? העברנו לו את:

1. מיקום ה-DLL שאנחנו רוצים לטעון
2. קובץ ה-lib שכולל את שמות הפונקציות

כתחליף ל-א', נשתמש בפונקציה LoadLibrary. הפונקציה מקבלת נתיב מלא הכולל שם של DLL, טוענת את ה-DLL לזיכרון ומחזירה מצביע על ה-DLL המבוקש (אם לא אירעה שגיאה).

כתחליף ל-ב', נשתמש בפונקציה GetProcAddress. כפי שאפשר להבין משמה, היא מחזירה את הכתובת בזיכרון בה נמצאת הפרוצדורה שביקשנו (בשפת אסמבלי, פרוצדורה הינה פונקציה).

```
#include "pch.h"
#define LIBRARY "c:\\Users\\barak\\source\\repos\\mydll\\Release\\mydll.dll"

typedef void(*PFUNC)(void);

int main()

{
    HMODULE hModule = LoadLibraryA(LIBRARY);
    if (NULL == hModule) {
        printf("Failed to load DLL\n");
        return 0;
    }
    PFUNC pFunc = (PFUNC)GetProcAddress(hModule, "Share");
    if (NULL != pFunc) {
        (*pFunc)();
    }
    else {
```

```
        printf("Failed to load function\n");
    }
    return 0;
}
```

שימו לב לטכניקה שבה אנחנו משתמשים בכתובת החזרה. אנחנו מודיעים לקומפיילר, באמצעות typedef, ש-PFUNC הוא מצביע לפונקציה, שמקבלת void ומחזירה void. זו צורת הכתיבה המקובלת ב-C.

כאשר אנחנו קוראים ל-GetProcAddress, אנחנו מבצעים casting של ערך החזרה: ערך החזרה הוא כאמור כתובת בזיכרון. ה-casting מסביר לקומפיילר, שהכתובת הזו היא מצביע לפונקציה, ושהפונקציה הזו מקבלת void ומחזירה void.

הנה, כך נראים המשתנים שהגדרנו, תוך כדי הרצה:

Locals

| Name | Value | Type |
|---|---|---|
| hModule | 0x0fee0000 {mydll.dll!_IMAGE_DOS_HEADER __Im... | HINSTANCE__ * |
| pFunc | 0x0fee1000 {mydll.dll!Share(void)} | void(*)() |

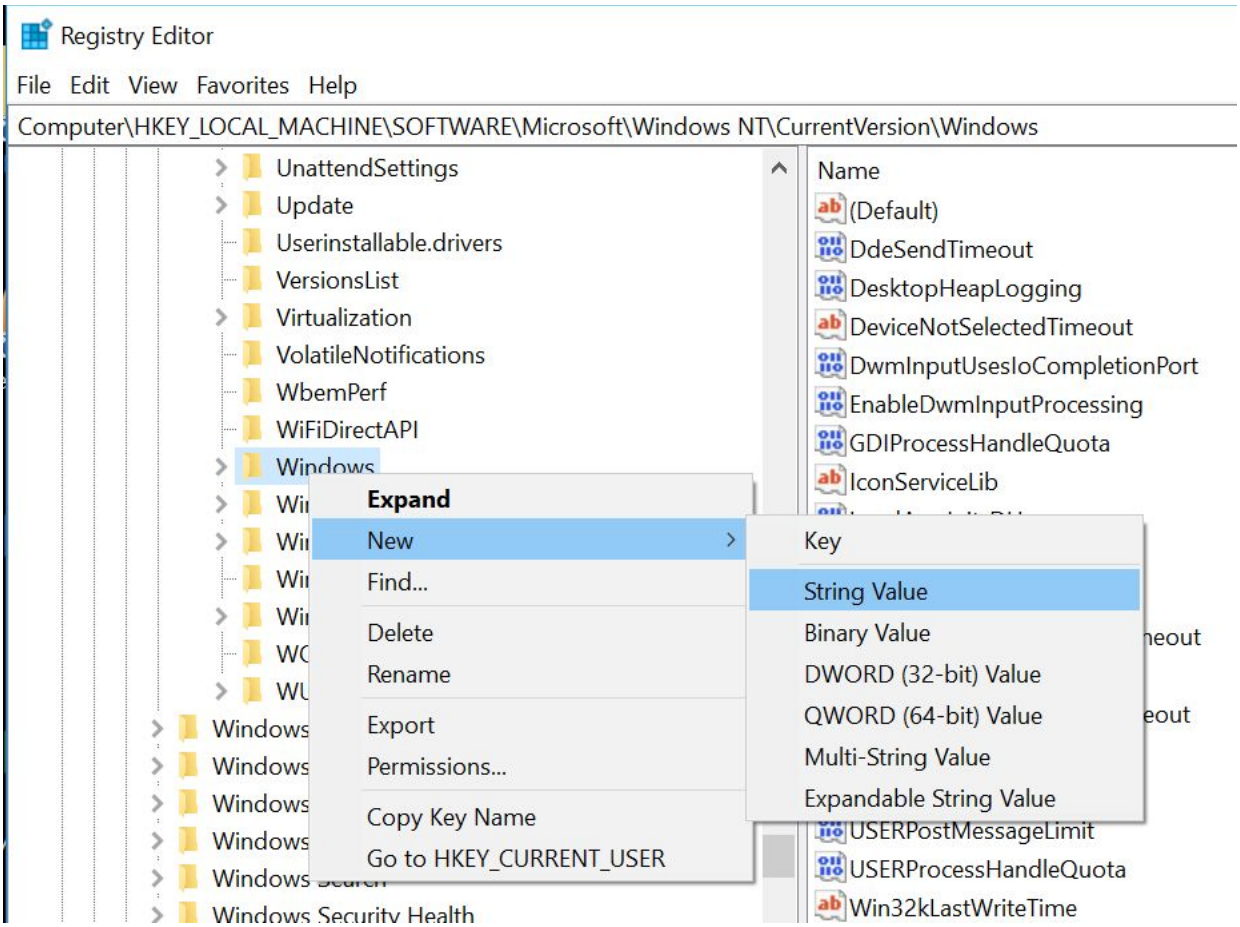
המשתנה hModule מקבל את הכתובת בזיכרון הוירטואלי של ה-process, כתובת שאליה נטען ה-DLL המבוקש. ה-DLL מתחיל בחלק שנקרא IMAGE_DOS_HEADER ולכן נראה ש-hModule מצביע עליו. המשתנה pFunc מקבל את הכתובת של הפונקציה Share מתוך mydll.dll. ניתן לראות שמיקום הפונקציה הוא כ-0x1000 בתים אחרי ההתחלה של ה-DLL, היכן שאזור הקוד בדרך כלל מתחיל.

<u>שימוש ברגיסטרי ל-DLL Injection</u>

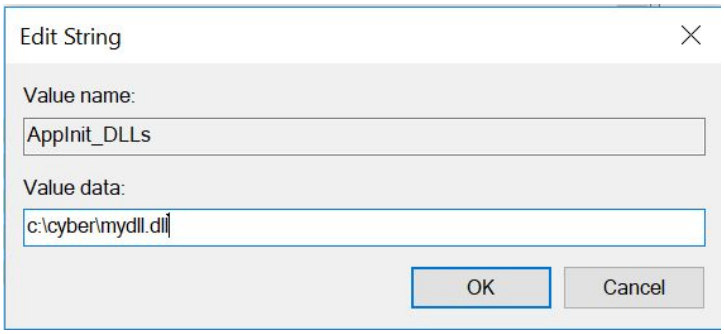
בחלק זה נלמד אודות AppInit וכיצד באמצעות שימוש בשדה זה אפשר להזריק את ה-DLL שלנו להיכן שנרצה. לאחר שלמדנו ליצור DLL, זהו חלק קל מאד.

פיתחו את Autoruns. אחד מהטאבים שלמעלה הוא AppInit. אנחנו מקווים שתחת הטאב הזה אין לכם כלום על המחשב, אם כן זה חשוד מאד ומומלץ לבדוק ב-Virus Total את ה-DLL שמופיע אצלכם. השדה הזה ברגיסטרי עובד כך: כל תוכנה שנטענת לזיכרון ושמשתמשת ב-user32.dll, בודקת את הערך של AppInit וטוענת כל DLL שנמצא שם. קשה לרדת לעומק דעתה של מייקרוסופט שיצרה את המנגנון הזה, אך הוא בהחלט שם וניתן להשתמש בו. כפי שראינו, יש אוסף לא קטן של processים שמריצים את user32.dll.

בחלק זה בתרגיל, שנו את קוד ה-DLL כך ש-dllmain יפתח חלון משלו ויכתוב הודעה לבחירתכם. כעת הכניסו את הערך לתוך הרגיסטרי במיקום **HKEY_LOCAL_MACHINE\Software\Microsoft\Windows NT\CurrentVersion\Windows**. כדי לבצע זאת, היכנסו לתיקיה הנ"ל, לחצו על קליק ימני ובחרו ליצור ערך רגיסטרי חדש:




שם הערך צריך להיות AppInit_DLLs, והערך שבתוכו צריך להיות ה-DLL שאתם מבקשים להריץ.




כדי שזה יעבוד, אתם צריכים לשנות שני ערכי רגיסטרי:

1. הערך של LoadAppInit_DLLs צריך להיות 1 (כלומר True)
2. הערך של RequireSignedAppinit_DLLs צריך להיות 0 (כלומר False), כיוון שה-DLLים שאנחנו מייצרים אינם חתומים על ידי יצרן תוכנה אותנטי

כעת פיתחו חלון Excel, Word לדוגמה, ותמצאו שהחלון שלכם צץ 😊

ביצוע DLL Injection בשיטה נוספת

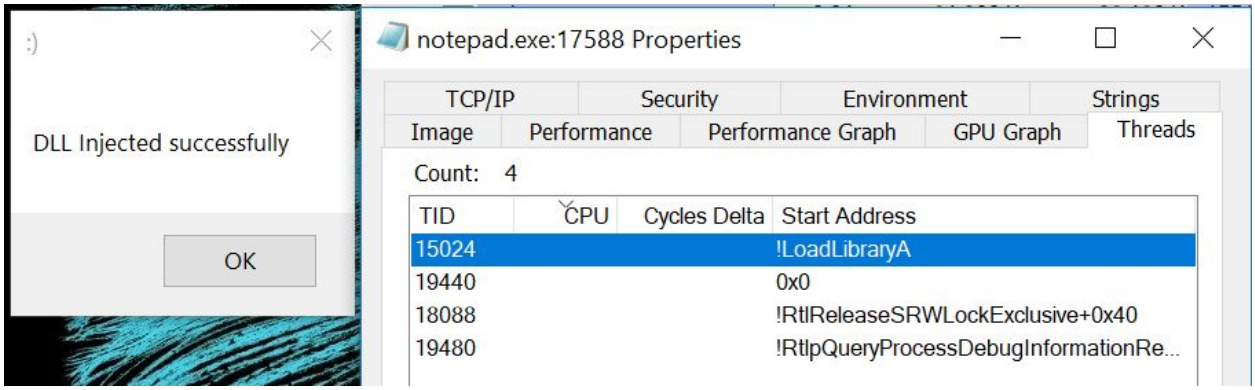
מוטי החל לעבוד בחברת תוכנה. לאחר זמן מה פנה אליו המנהל שלו וסיפר לו שלאחת התוכנות שהחברה פיתחה צריך להוסיף מסך פתיחה עם שם החברה, אולם קוד המקור של התוכנה אבד. "אין בעיה" אמר מוטי "אשתמש ב-DLL injection להוספת מסך פתיחה כפי שביקשת". לאחר זמן קצר הציג מוטי את התוצאה למנהל שלו וזכה להערכה רבה.

בחלק זה נבצע DLL injection בשיטה שנבחרה על ידי מוטי. להלן הסבר כללי על השיטה, את הפרטים תצטרכו להשלים לבד :)

1. נבחר process קיים (מרוחק, כלומר לא בן של ה-process שלנו) ונפתח אותו
2. ב-process המרוחק, ניצור thread חדש
3. כזכור כאשר פותחים thread חדש, מעבירים לו את הפונקציה שממנה יש להתחיל את הריצה. במקרה שלנו, נעביר לו את הכתובת של LoadLibrary, יחד עם הנתיב של ה-DLL שאותו ברצוננו לטעון
4. שלב 3 מעלה בפנינו שני קשיים:
   a. אנחנו צריכים לדעת מה הכתובת של LoadLibrary
   b. אנחנו צריכים שהמחרוזת שמתארת את הנתיב של ה-DLL תהיה שמורה בזיכרון של ה-process המרוחק (העובדה שהמחרוזת שמורה בזיכרון של ה-process שלנו לא מספיק טובה, מכיוון שאיננו חולקים זיכרון וירטואלי עם ה-process המרוחק)
5. כדי לפתור את הקושי הראשון, נבדוק מה הכתובת של LoadLibrary ב-process שלנו. הכתובת תהיה זהה לכתובת של LoadLibrary ב-process המרוחק, מכיוון שהפונקציה הזו נמצאת בתוך DLL שמשותף לכל ה-processים (איזה DLL? מיצאו אותו באמצעות MSDN)
6. כדי לפתור את הקושי השני, נקצה זיכרון ב-process המרוחק ונשמור לתוכו את המחרוזת עם שם ה-DLL שעליו להעביר ל-LoadLibrary

שימו לב שה-DLL וקוד התוכנית שמזריקה אותו צריכים להיות באותה ארכיטקטורה (32 / 64 ביט) של התוכנית אליה ה-DLL מוזרק, אחרת הפונקציה CreateRemoteThread תחזור עם קוד שגיאה Access_Denied.

דוגמה לפעולת ההזרקה: במסך ה-process explorer אפשר לראות של-notepad.exe יש thread שנקודת ההתחלה שלו היא LoadLibrary. לצד זה, מופיע חלון שהוקפץ על ידי notepad.exe.



לפניכם שלד של תוכנית שמבצעת DLL Injection לפי השלבים שהוסברו. התוכנית כוללת תיעוד של השלבים לפי סדרם. חלקים מהקוד הושמטו ובמקומם יש שלוש נקודות. השלימו את החסר, וגירמו ל-notepad.exe להריץ חלון נוסף עם הודעה אישית ממכם!

```
// Barak Gonen 2019
// Skeleton code - inject DLL to a running process

#include "pch.h"

int main()
{
    // Get full path of DLL to inject
    DWORD pathLen = GetFullPathNameA(...);

    // Get LoadLibrary function address –
    // the address doesn't change at remote process
    PVOID addrLoadLibrary = (PVOID)GetProcAddress(GetModuleHandle(...),
                            LoadLibraryA");

    // Open remote process
    proc = OpenProcess(...);

    // Get a pointer to memory location in remote process,
    // big enough to store DLL path
    PVOID memAddr = (PVOID)VirtualAllocEx(...);
    if (NULL == memAddr) {
        err = GetLastError();
        return 0;
    }

    // Write DLL name to remote process memory
    check = WriteProcessMemory(...);
    if (0 == check) {
        err = GetLastError();
        return 0;
    }

    // Open remote thread, while executing LoadLibrary
    // with parameter DLL name, will trigger DLLMain
    HANDLE hRemote = CreateRemoteThread(...);
    if (NULL == hRemote) {
        err = GetLastError();
        return 0;
    }

    WaitForSingleObject(hRemote, INFINITE);
    check = CloseHandle(hRemote);
    return 0;
}
```

סיימנו את הבסיס של DLLים והזרקת DLLים. ישנן עוד שיטות רבות להזריק קוד שלנו לתוך Process קיים, יש לכם את הבסיס ללמוד אותן. כעת אתם חופשיים לשחק בפארק בכוחות עצמכם :)